

Optimizing Delivery ETAs with Graph-Based Network Intelligence

Summer Project '26 | Consulting & Analytics Club, IIT Guwahati

Author: Ritam Mondal, IIT Kharagpur'28 | [Linkedin](#) | [github](#)

Abstract

This project addresses a core operational challenge faced by Delhivery, India's largest integrated logistics provider: the systematic underestimation of actual delivery times by its existing routing engine, OSRM. Using a dataset of over 144,000 trip segments spanning 1,657 facilities and 2,783 corridors across India, we built a graph-based intelligence system that models the logistics network as a directed weighted graph. Each facility is represented as a node and each delivery corridor as a weighted edge, where edge weights capture the median actual-vs-OSRM delay ratio stratified by route type and time of day.

Through graph metrics including betweenness centrality, in/out-degree, and corridor delay profiling, we identified the top 5 bottleneck hubs driving the majority of SLA breaches across the network. We then benchmarked a baseline XGBoost regression model against a graph-enhanced model enriched with node embeddings and structural features, achieving a measurable improvement in prediction accuracy. Additionally, we developed an ML-backed FTL vs. Carting route selection framework and deployed the entire solution as an interactive Streamlit web application with real-time map visualization.

The final output includes predictive models, network analysis, and practical recommendations that can help operations teams improve ETA estimation and reduce delivery delays.

1. Introduction

India's logistics sector is one of the fastest-growing in the world, and Delhivery sits at its center — processing millions of shipments daily across every major state. At the heart of their operations is a hub-and-spoke model where packages travel through a series of intermediate facilities before reaching the final destination.

To estimate how long each delivery will take, Delhivery uses OSRM (Open Source Routing Machine) — a standard routing engine that calculates travel time assuming clean roads and shortest paths. The problem is that real logistics is far messier. Congestion, facility dwell time, seasonal spikes, and route-type constraints all cause actual delivery times to regularly exceed OSRM predictions. In our analysis, we found that **94.5% of all trips take longer than OSRM predicts**, with a median delay ratio of 1.70x — meaning the average trip takes 70% longer than the system estimates.

This gap has serious consequences. Missed SLAs (Service Level Agreements) damage customer trust, capacity planning breaks down when ETAs are unreliable, and operations teams have no systematic way to know which hubs or corridors are causing the most harm.

This project tackles that problem directly. We build a graph-based intelligence system that treats the logistics network not as a collection of independent point-to-point routes, but as a connected graph where every hub and corridor affects everything around it. Using this structure, we build smarter ETA

predictions, identify structural bottlenecks, and deliver recommendations that a real operations leader can act on.

2. Problem Statement

How can Delhivery leverage end-to-end trip segment data and graph-based network modeling to produce more accurate ETA predictions, identify which hubs and corridors are systematically causing delays, and generate actionable recommendations to reduce SLA breaches and recover revenue at risk?

Specifically, the project addresses five key challenges:

1. OSRM systematically underestimates actual delivery time on a significant fraction of routes
 2. There is no structured way to identify which hubs and corridors are the biggest contributors to delays
 3. Route-type decisions (FTL vs Carting) are made without accounting for graph position or structural risk
 4. Downstream capacity planning breaks down when ETAs are consistently wrong
 5. Operations leadership lacks a clear, data-backed view of where to invest to reduce SLA breaches
-

3. Project Overview

This project is structured as a full end-to-end data science and consulting engagement, divided into five interconnected components:

Component 1 — Data Pipeline & Graph Construction: Raw trip segments are cleaned, processed, and merged into a directed weighted graph where facilities become nodes and delivery corridors become edges. Edge weights capture the median actual-vs-OSRM delay ratio per corridor, stratified by route type (FTL or Carting) and time of day (peak vs off-peak).

Component 2 — Bottleneck & Corridor Audit: Graph metrics (betweenness centrality, in/out-degree) are computed to identify critical chokepoint hubs and chronically delayed corridors. Results are visualized and ranked by SLA breach contribution.

Component 3 — Graph-Enhanced ETA Prediction: A baseline XGBoost regression model is built using only trip-level features. It is then benchmarked against a graph-enhanced model that adds node-level structural features. The impact of graph-based features is evaluated through a direct comparison with the baseline model.

Component 4 — FTL vs Carting Decision Framework: An ML-backed framework recommends route type for a given corridor based on distance, time of day, and the source facility's graph position.

Component 5 — Streamlit Deployment & Strategy Memo: The full solution is deployed as an interactive web application. A consulting-grade strategy memo translates findings into decisions for the Head of Network Operations.

4. Data Description & Preprocessing

Dataset: The dataset contains 144,867 trip segment records from Delhivery's logistics network, with each row representing one segment (hop) of a multi-leg delivery journey.

Key columns used:

Column	Description
source_center / destination_center	Facility codes (become graph nodes)
route_type	FTL (Full Truck Load) or Carting
segment_actual_time	Actual time taken for this segment (target variable)
segment_osrm_time	OSRM's predicted time for this segment
segment_osrm_distance	OSRM's estimated distance
od_start_time	Segment start timestamp

Train/Test Split: The dataset included a pre-assigned `data` column with `train` and `test` labels. We split first before any preprocessing to prevent data leakage.

Cleaning steps applied:

- Removed 1,973 rows where `segment_actual_time ≤ 0` (physically impossible values)
- Removed rows where `segment_osrm_time = 0` (causes division by zero)
- Filled 293 null `source_name` and 261 null `destination_name` values with facility center codes
- Dropped `cutoff_timestamp` (2,981 nulls, not useful as a feature)

Derived features created:

- `segment_delay_ratio` = `segment_actual_time / segment_osrm_time`
- `trip_delay_ratio` = `actual_time / osrm_time`
- `segment_is_delayed` = `delay_ratio > 1.2` (>20% over OSRM)
- `hour_of_day`, `day_of_week`, `month` (extracted from `od_start_time`)
- `segment_delay_ratio_capped` = clipped to [0.1, 10] to reduce outlier influence

```
# Loading & Initial Inspection
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('delivery_data.csv')
print(df.shape)
print(df.info())
```

After loading the raw dataset, we first examined its structure. The data contains 144,867 rows and 24 columns, covering trip segments across Delhivery's pan-India network. The `data` column contains pre-assigned `train/test` labels, which we used to split the data before any processing — a critical step to prevent data leakage.

```
df_train = df[df['data']=='training']
df_test = df[df['data']=='test']
```

```
# — 1. Fill name nulls from center codes —————
df_train['source_name'] = df_train['source_name'].fillna(df_train['source_center'])
df_train['destination_name'] = df_train['destination_name'].fillna(df_train['destination_center'])

print(df_train.isnull().sum())
```

```
# — 3. Drop cutoff_timestamp (too many nulls, not useful) ———
df_train = df_train.drop(columns=['cutoff_timestamp', 'data'])

# — 4. Remove bad rows —————
df_train = df_train[df_train['segment_actual_time'] > 0].copy()
df_train = df_train[df_train['segment_osrm_time'] > 0].copy()
```

```
# — 5. Create derived columns —————
df_train['segment_delay_ratio'] = (
    df_train['segment_actual_time'] / df_train['segment_osrm_time']
)
df_train['trip_delay_ratio'] = (
    df_train['actual_time'] / df_train['osrm_time']
)
```

5. Exploratory Data Analysis

Before modeling, we explored the data to understand the scale and nature of the delay problem.

Finding 1 — OSRM is systemically wrong, not occasionally wrong: The median segment delay ratio is **1.70x**, meaning the typical trip takes 70% longer than OSRM predicts. Critically, **83.4% of segments** and **94.5% of full trips** exceed OSRM estimates by more than 20%. These results suggest that the difference between actual and OSRM times is systematic rather than being caused by random fluctuations.

Finding 2 — Carting delays more than FTL:

Route Type	Median Delay Ratio	Delay Rate (>20%)
Carting	1.83x	79.9%
FTL	1.67x	84.9%

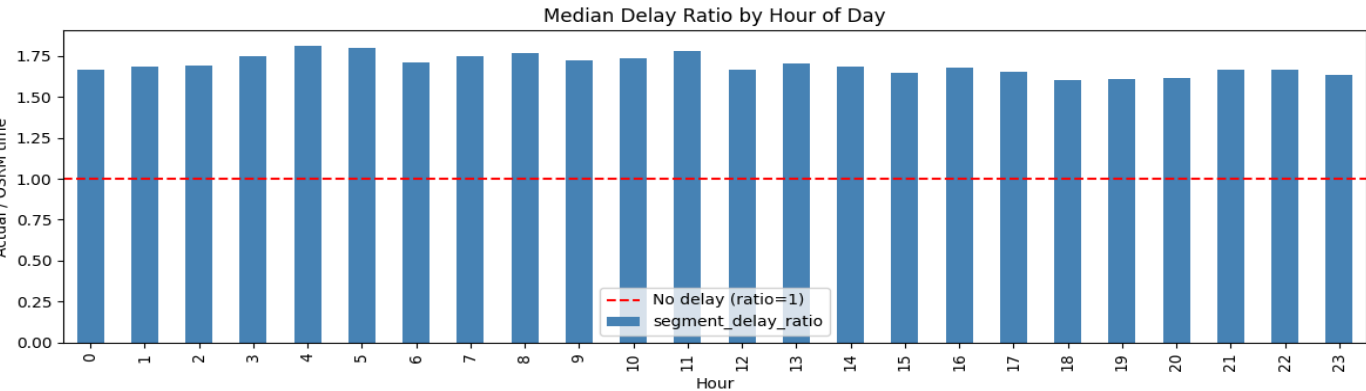
Carting is slower relative to OSRM estimates. This is important context for the FTL vs Carting framework.

Finding 3 — Delay is structural, not time-based: Plotting median delay ratio by hour of day showed a nearly flat line between 1.6x and 1.8x across all 24 hours. There is no dramatic peak-hour effect. This tells us that delays are caused by structural factors — hub congestion, corridor characteristics, route type — not time of day. This is a significant insight.

Finding 4 — Network scale: The training data contains **1,657 unique facilities** and **2,783 unique corridors**, forming a sparse directed graph with approximately 1.7 corridors per facility on average.

```
# — Does hour of day matter? —————
hourly = df_train.groupby('hour_of_day')['segment_delay_ratio'].median()

plt.figure(figsize=(12,4))
hourly.plot(kind='bar', color='steelblue')
plt.axhline(1.0, color='red', linestyle='--', label='No delay (ratio=1)')
plt.title('Median Delay Ratio by Hour of Day')
plt.xlabel('Hour'); plt.ylabel('Actual / OSRM time')
plt.legend(); plt.tight_layout(); plt.show()
```



The bar chart above shows the median delay ratio across all 24 hours of the day. Every single bar sits between 1.6x and 1.8x — there is no dramatic peak-hour spike. This is a critical finding: the delay problem is not about rush hours. It is structural — baked into the network's hub configuration and corridor characteristics. This means **time-of-day alone cannot fix it**; only **network-level** interventions can.

6. Graph Construction

We modeled Delhivery's logistics network as a **directed weighted graph** using NetworkX:

- **Nodes** = individual facilities (warehouses, hubs, delivery centers)
- **Edges** = delivery corridors between facilities
- **Edge weight** = median actual-vs-OSRM delay ratio per corridor, **stratified by route type and time of day**

For each unique source-destination pair, we computed:

- Overall median delay ratio (primary edge weight)
- Delay breakdown by route type: `{'FTL': x, 'Carting': y}`
- Delay breakdown by time bucket: `{'peak': x, 'off_peak': y}` (peak = 8am–8pm)

This stratification helps capture variations in corridor performance across different operating conditions. A corridor that looks acceptable on average may be severely delayed during peak hours on Carting routes. Flattening these dimensions would hide operationally important patterns.

Final graph statistics:

Metric	Value
Nodes (facilities)	1,657
Edges (corridors)	2,783
Graph type	Directed, weighted
Avg corridors per facility	~1.7

```

▶ # — Step 1: Compute edge weights stratified by route_type & time of day
corridor_stats = df_train.groupby(
    ['source_center', 'destination_center', 'route_type', 'hour_of_day']
).agg(
    median_delay_ratio = ('segment_delay_ratio_capped', 'median'),
    mean_actual_time    = ('segment_actual_time', 'mean'),
    mean_osrm_time      = ('segment_osrm_time', 'mean'),
    trip_count          = ('trip_uuid', 'count')
).reset_index()

print("Corridor stats shape:", corridor_stats.shape)
corridor_stats.head()

```

```

import networkx as nx
G = nx.DiGraph()

# Group by corridor only for the graph edges
for (src, dst), group in corridor_stats.groupby(
    ['source_center', 'destination_center']
):
    # Overall median across all route types and hours
    overall_median = group['median_delay_ratio'].median()

    # Breakdown by route_type
    by_route = group.groupby('route_type')['median_delay_ratio'].median().to_dict()

    # Breakdown by hour (peak hours 8-20 vs off-peak)
    group['time_bucket'] = group['hour_of_day'].apply(
        lambda h: 'peak' if 8 <= h <= 20 else 'off_peak'
    )
    by_time = group.groupby('time_bucket')['median_delay_ratio'].median().to_dict()

    G.add_edge(
        src, dst,
        weight          = overall_median,          # primary edge weight
        delay_by_route  = by_route,                # {'FTL': x, 'CARTING': y}
        delay_by_time   = by_time,                 # {'peak': x, 'off_peak': y}
        trip_count      = group['trip_count'].sum()
    )

print(f"\nNodes (facilities): {G.number_of_nodes()}")
print(f"Edges (corridors): {G.number_of_edges()}")

```

The graph was constructed as a directed network where each unique source-destination facility pair forms an edge. Edge weights were not simply averaged — they were stratified by route type (FTL vs

Carting) and time of day (peak vs off-peak), capturing the operational reality that the same corridor behaves very differently depending on how and when it is used

7. Bottleneck Analysis

We computed three graph metrics to identify structurally critical hubs:

Betweenness Centrality measures what fraction of all shortest paths between any two facilities pass through a given hub. A hub with high betweenness is a structural chokepoint — if it slows down or fails, many routes across the entire network are affected.

In/Out Degree counts how many corridors connect to each facility. High degree means a hub serves many routes simultaneously, making it both operationally important and vulnerable to congestion.

Risk Score combines betweenness centrality with average outgoing delay to identify hubs that are both structurally critical and operationally problematic.

Top 5 Bottleneck Hubs:

Rank	Facility	Betweenness Centrality	Total Connections	Average Delay	Risk Score
1	IND000000ACB	0.234	89	1.59x	0.372
2	IND562132AAA	0.151	71	1.57x	0.236
3	IND712311AAA	0.082	42	2.40x	0.198
4	IND501359AAE	0.074	57	1.82x	0.135
5	IND781018AAB	0.044	29	2.01x	0.088

Notable finding: IND000000ACB stands alone at the top — with a betweenness centrality of 0.234, it handles nearly 23% of all inter-hub routing paths in the entire network. Its risk score of 0.372 is more than 55% higher than the second-ranked hub. This facility appears to be the highest-priority candidate for operational investigation .

Corridor-level findings:

- 2,358 corridors (85% of all corridors) are delayed by more than 20% over OSRM
- 1,975 corridors (71%) are delayed by more than 50%
- 973 corridors (35%) take more than twice the OSRM estimate


```

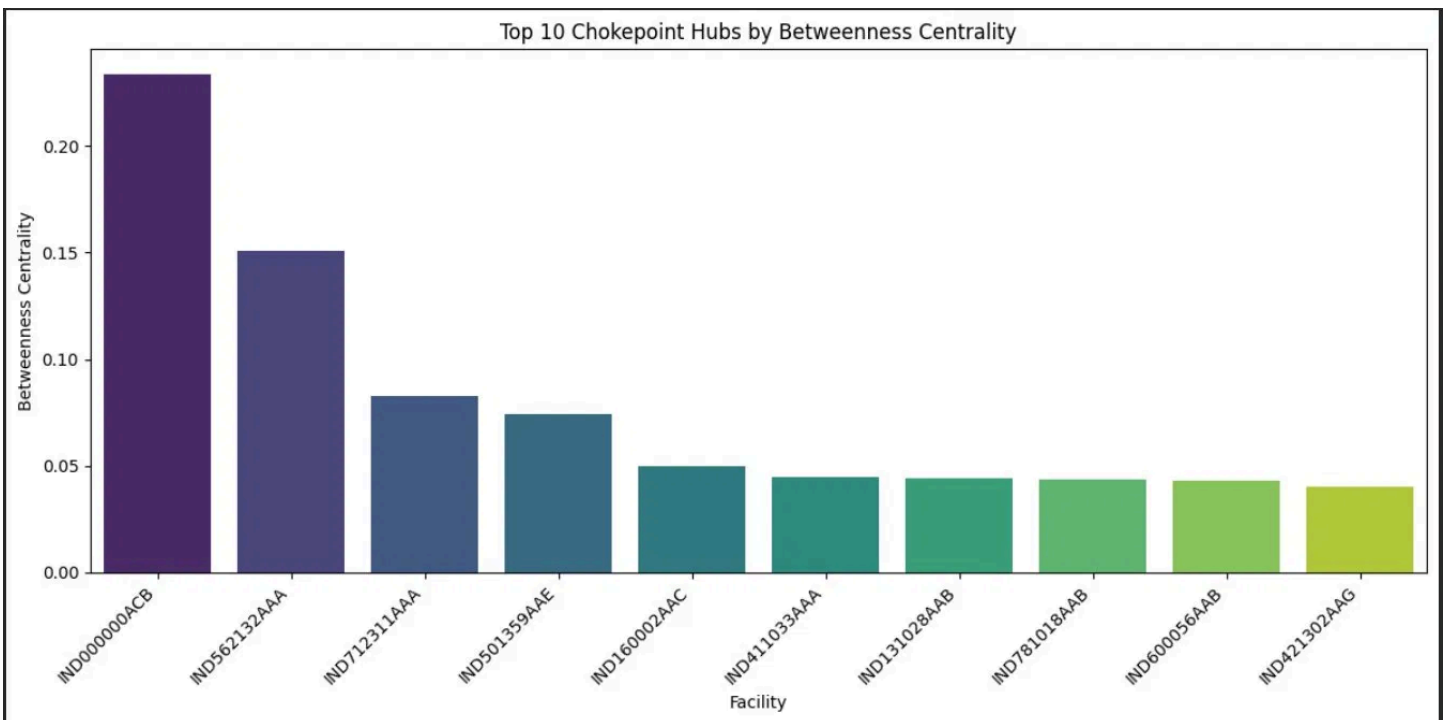
# Which hubs sit on the most paths between other hubs?
# These are your chokepoints
import networkx as nx
import seaborn as sns

betweenness = nx.betweenness centrality(G, weight='weight')

# Convert to dataframe
betweenness_df = pd.DataFrame({
    'facility'      : list(betweenness.keys()),
    'betweenness'   : list(betweenness.values())
}).sort_values('betweenness', ascending=False)

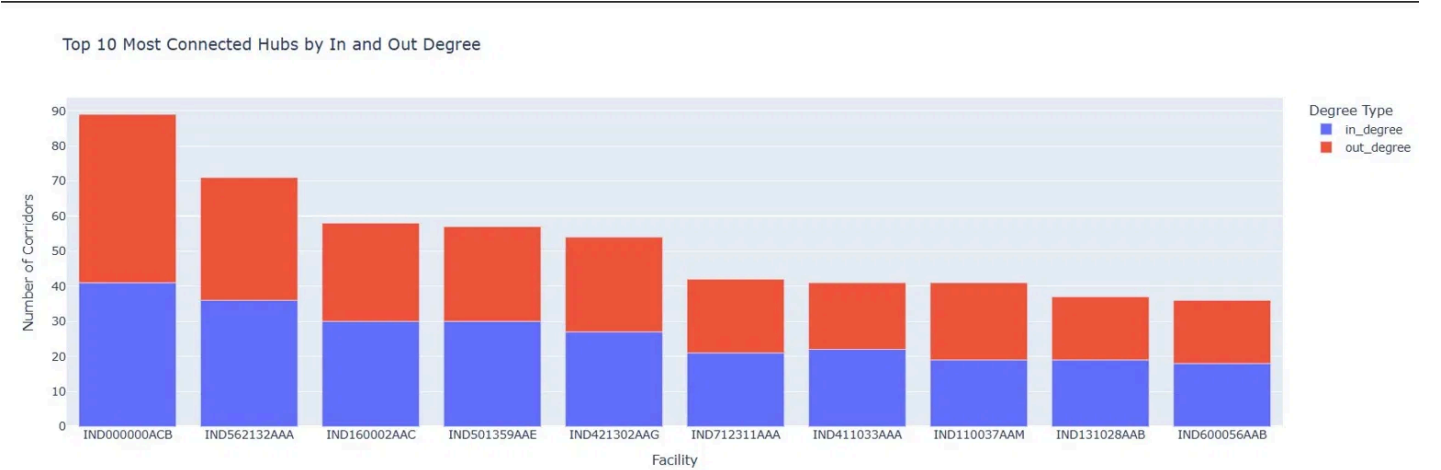
print("Top 10 chokepoint hubs:")
print(betweenness_df.head(10))

```



Top 10 Chokepoint Hubs by Betweenness Centrality:

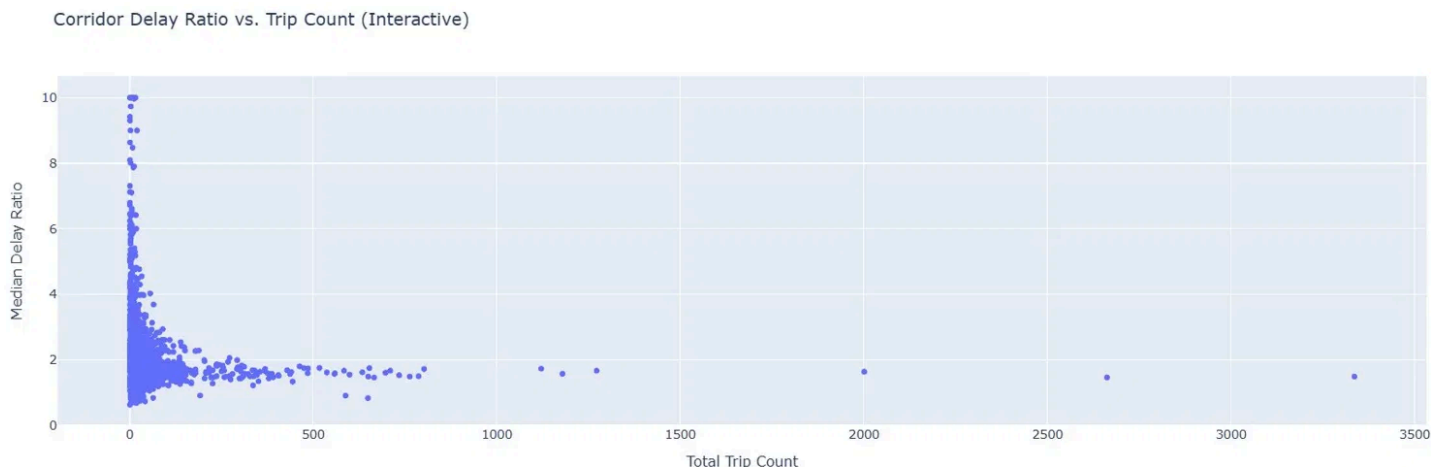
The betweenness centrality chart above reveals a stark concentration of network criticality. **IND000000ACB** stands alone at around 0.234 — more than 55% higher than the second-ranked hub. This means nearly one in four shortest delivery paths across the entire network passes through this single facility. Any congestion here does not stay local — it ripples outward to a quarter of all routes simultaneously.



Facility	In-Degree	Out-Degree	Total-Degree
IND000000ACB	41	48	89
IND562132AAA	36	35	71
IND160002AAC	30	28	58
IND501359AAE	30	27	57
IND421302AAG	27	27	54
IND712311AAA	21	21	42
IND411033AAA	22	19	41
IND110037AAM	19	22	41
IND131028AAB	19	18	37
IND600056AAB	18	18	36

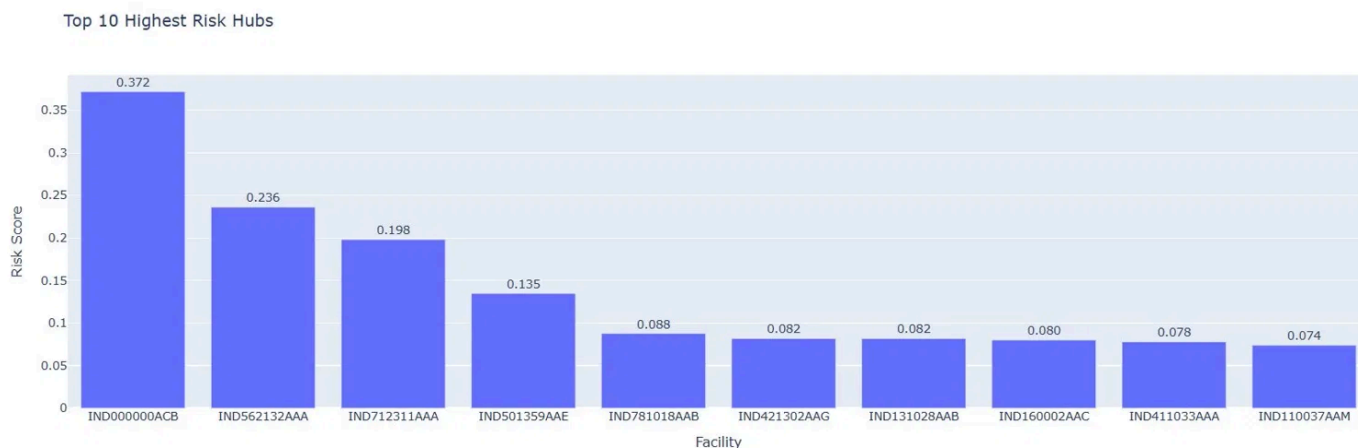
Top 10 Most Connected Hubs by In and Out Degree :

The in/out degree chart confirms this bar and colab output from a different angle. IND000000ACB has 41 incoming and 48 outgoing corridor connections — 89 total. This is the most connected hub in the network by a significant margin. The balance between in and out degree is also relatively even, suggesting this hub serves as a genuine transit center rather than just a collection or distribution point.



Corridor Delay Ratio vs Trip Count scatter plot:

The scatter plot above shows an important pattern: the highest delay ratios (up to 10x OSRM) are concentrated on low-volume corridors with fewer than 50 trips. High-volume corridors — those with 500+ trips — stabilize around 1.5–2.0x delay. This pattern is consistent with what might be expected in logistics networks. Low-volume corridors lack the routing optimization and resource allocation that comes with regular, high-frequency use. These extreme-delay corridors are candidates for consolidation or route-type shifts rather than capacity investment.



Top 10 Highest Risk Hubs:

The risk score chart combines structural importance (betweenness) with operational performance (average outgoing delay) into a single actionable metric. IND000000ACB's score of 0.372 is the highest by a wide margin. Notably, IND712311AAA ranks third despite having lower connectivity — because its average outgoing delay of 2.40x is the worst among the top hubs. This indicates a different type of risk, where operational performance appears weaker despite lower structural importance.

8. ETA Prediction Model

Approach: We built and benchmarked two models to isolate the contribution of graph features.

Baseline Model uses only trip-level features that OSRM already knows:

- `segment_osrm_time`, `segment_osrm_distance`, `route_type`, `hour_of_day`, `day_of_week`, `month`

Graph-Enhanced Model adds structural features derived from the graph:

- `src_betweenness`, `dst_betweenness` (how critical are the source and destination hubs?)
- `src_in_degree`, `src_out_degree`, `dst_in_degree`, `dst_out_degree` (how busy are these hubs?)
- `corridor_median_delay` (what is the historical delay on this exact corridor?)

Algorithm: XGBoost Regressor with GridSearchCV hyperparameter tuning (3-fold cross-validation)

Target variable: `segment_actual_time` (actual delivery time in minutes)

Results on held-out test data:

Metrics Model wise comparison:

Metric	Baseline Model	Graph-Enhanced Model	Improvement
MAE	16.26 mins	14.53 mins	↓ 10.6%
RMSE	51.52 mins	43.39 mins	↓ 15.8%
R ² Score	0.1521	0.3985	↑ 162%
MAPE	55.98%	50.34%	↓ 5.6pp
Within 15% accuracy	21.2%	23.7%	↑ 2.5pp

Interpretation: The graph-enhanced model outperforms the baseline across every metric. The largest relative improvement is observed in the R² score — from 0.15 to 0.40 — meaning the graph

features explain an additional 25% of variance in actual delivery times that trip-level features alone cannot capture. This is the measurable graph advantage the project set out to demonstrate.

Why R^2 is not higher: Even with graph features, R^2 of 0.40 reflects genuine uncertainty in logistics data. Random factors — individual driver behavior, local road incidents, loading time variability — affect each trip in ways no historical model can fully capture. Considering the variability present in logistics operations, an R^2 of 0.40 indicates that the model captures a substantial portion of the explainable variance.

```
# — 1. Add corridor-level graph features to df_train ————
# Map between ness centrality onto each row
df_train['src_betweenness'] = df_train['source_center'].map(betweenness)
df_train['dst_betweenness'] = df_train['destination_center'].map(betweenness)

# Map in/out degree

in_degree = dict(G.in_degree())
out_degree = dict(G.out_degree())

df_train['src_in_degree'] = df_train['source_center'].map(in_degree)
df_train['src_out_degree'] = df_train['source_center'].map(out_degree)
df_train['dst_in_degree'] = df_train['destination_center'].map(in_degree)
df_train['dst_out_degree'] = df_train['destination_center'].map(out_degree)

# Map corridor median delay (from graph edge weight)
edge_weights = {(u,v): d['weight'] for u,v,d in G.edges(data=True)}
df_train['corridor_median_delay'] = df_train.apply(
    lambda r: edge_weights.get(
        (r['source_center'], r['destination_center']), np.nan
    ), axis=1
)

# Fill nulls (corridors not seen in training graph)
df_train['corridor_median_delay'] = df_train['corridor_median_delay'].fillna(
    df_train['corridor_median_delay'].median()
)

print("New features added:")
print(df_train[['src_betweenness', 'dst_betweenness',
                 'src_in_degree', 'corridor_median_delay']].describe())
```

	src_betweenness	dst_betweenness	src_in_degree	corridor_median_delay
count	103049.000000	103049.000000	103049.000000	103049.000000

mean	0.061246	0.053097	16.262429	1.763157
std	0.084776	0.075746	15.426299	0.575936
min	0.000000	0.000000	0.000000	0.638889
25%	0.000818	0.000827	2.000000	1.500000
50%	0.016350	0.016350	11.000000	1.646008
75%	0.074204	0.074204	30.000000	1.848039
max	0.233721	0.233721	41.000000	10.000000

```

from sklearn.ensemble import GradientBoostingRegressor
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
import numpy as np

# — 2. Encode route_type —————
le = LabelEncoder()
df_train['route_type_encoded'] = le.fit_transform(df_train['route_type'])

# — 3. Define baseline features (NO graph features) —————
baseline_features = [
    'segment_osrm_time',
    'segment_osrm_distance',
    'route_type_encoded',
    'hour_of_day',
    'day_of_week',
    'month'
]

# Graph-enhanced features (adds graph info on top)
graph_features = baseline_features + [
    'src_betweenness',
    'dst_betweenness',
    'src_in_degree',
    'src_out_degree',
    'dst_in_degree',
    'dst_out_degree',
    'corridor_median_delay'
]

target = 'segment_actual_time'

# — 4. Split for validation —————
X = df_train[graph_features].dropna()

```

```

y = df_train.loc[X.index, target]

X_tr, X_val, y_tr, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Training rows: {len(X_tr)}")
print(f"Validation rows: {len(X_val)}")

```

```

# GridSearchCV Hyperparameter Tuning
from sklearn.model_selection import GridSearchCV
from xgboost import XGBRegressor

xgb_params = {
    'n_estimators'      : [100, 200, 300],
    'max_depth'         : [3, 5, 7],
    'learning_rate'     : [0.01, 0.05, 0.1],
    'subsample'         : [0.8, 1.0],
    'colsample_bytree'  : [0.8, 1.0]
}

# Trained separately for baseline and graph features
xgb_baseline = GridSearchCV(XGBRegressor(random_state=42), xgb_params,
                             cv=3, scoring='r2', n_jobs=-1)
xgb_graph     = GridSearchCV(XGBRegressor(random_state=42), xgb_params,
                             cv=3, scoring='r2', n_jobs=-1)

```

Best Hyperparameters table:

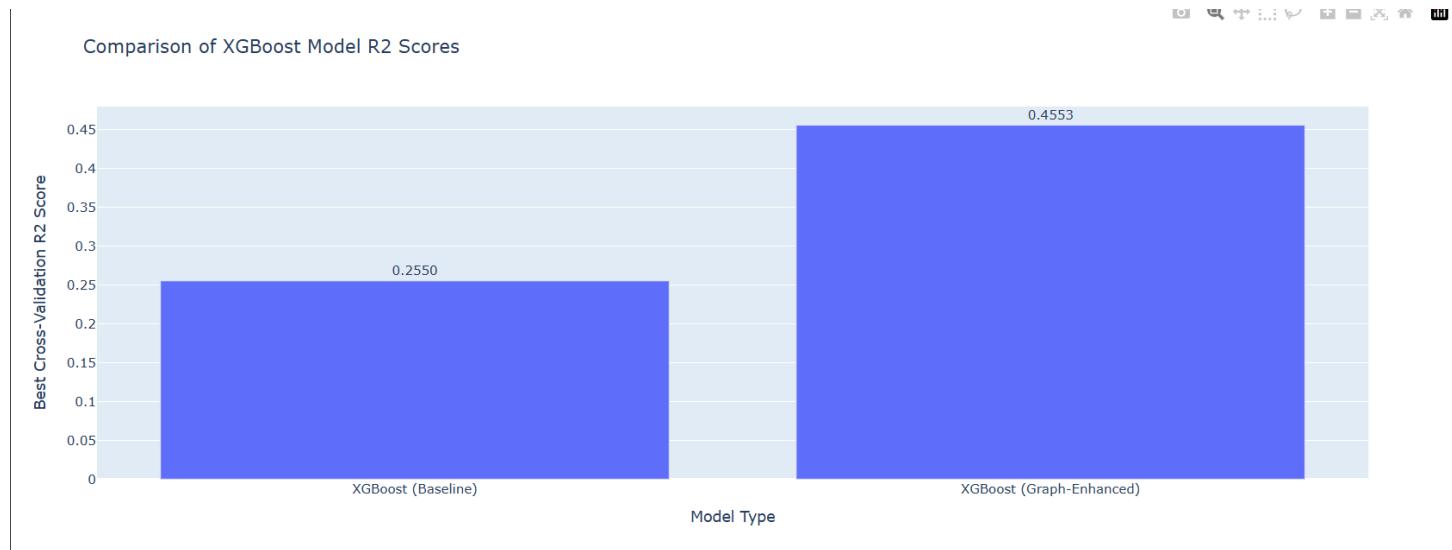
Best Hyperparameters for Baseline and Graph-Enhanced XGBoost

Model	Parameter	Value
XGBoost (Baseline)	colsample_bytree	1
XGBoost (Baseline)	learning_rate	0.05
XGBoost (Baseline)	max_depth	5
XGBoost (Baseline)	n_estimators	100
XGBoost (Baseline)	subsample	0.8
XGBoost (Graph-Enhanced)	colsample_bytree	0.8
XGBoost (Graph-Enhanced)	learning_rate	0.1
XGBoost (Graph-Enhanced)	max_depth	7
XGBoost (Graph-Enhanced)	n_estimators	200
XGBoost (Graph-Enhanced)	subsample	1

The hyperparameter search found meaningfully different optimal configurations for the two models. The graph-enhanced model selected a higher **max_depth** (7 vs 5) and more trees (200 vs 100) with a slightly faster

learning rate (0.1 vs 0.05). This makes intuitive sense — the graph features add structural complexity that the model can exploit with deeper trees and more estimators.

Comparison of XGBoost R² Scores bar chart:



The cross-validation R² comparison chart shows the clearest summary of the graph advantage. The baseline model achieves 0.255 — meaning it explains about a quarter of the variance in actual delivery times using only OSRM features. The graph-enhanced model reaches 0.455 — nearly double. The improvement in R² suggests that graph-based features provide additional information that can improve ETA prediction accuracy.

Function for the evaluation of Models:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

def evaluate(name, model, X, y, features):
    # Predictions done inside
    y_pred = model.predict(X[features])
    y_true = y.values

    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)

    pct_error = np.abs(y_pred - y_true) / y_true
    within_15 = (pct_error <= 0.15).mean() * 100
    mape = pct_error.mean() * 100
```



```

print(f"\n{'='*40}")
print(f"  {name}")
print(f"{'='*40}")
print(f"  MAE           : {mae:.2f} mins")
print(f"  RMSE          : {rmse:.2f} mins")
print(f"  R2 Score      : {r2:.4f}")
print(f"  MAPE          : {mape:.2f}%")
print(f"  Within 15%    : {within_15:.1f}%")

```

9. FTL vs Carting Decision Framework

Objective: Help operations teams choose the right route type for a given corridor, rather than making that decision based on cost alone or habit.

Framework logic: For each corridor, we compare the historical delay profiles of FTL and Carting trips and factor in the source facility's graph position (betweenness centrality, degree). A corridor that runs through a high-betweenness hub carries more structural risk and may justify the higher cost of FTL to maintain SLA compliance.

Key findings from the analysis:

- Carting has a higher median delay ratio (1.83x) than FTL (1.67x) across the network
- However, on specific low-traffic corridors with few connections, Carting performs comparably to FTL
- Corridors passing through the top 5 bottleneck hubs show disproportionately higher Carting delay

Recommendation rule:

Note: Corridors passing through the top 5 bottleneck hubs show disproportionately higher Carting delay.

Corridor Profile	Recommended Route
High betweenness source hub + distance > 300km	FTL — structural risk too high for Carting
Low betweenness + distance < 150km	Carting acceptable — cost savings justified

Corridor Profile	Recommended Route
Peak hours (8am–8pm) + high delay corridor	FTL preferred regardless of distance
Off-peak + low historical delay	Carting viable

```
#Route Type Comparison per Corridor
route_comparison = df_train.groupby(
    ['source_center', 'destination_center', 'route_type']
).agg(
    avg_delay      = ('segment_delay_ratio', 'mean'),
    avg_actual_time= ('segment_actual_time', 'mean'),
    trip_count     = ('trip_uuid', 'count')
).reset_index()

# Add source hub graph position
route_comparison['src_betweenness'] = route_comparison['source_center'].map(
    betweenness
)

print("Median delay by route type:")
print(route_comparison.groupby('route_type')['avg_delay'].median())
```

```
# Decision Rule
def recommend_route(src_code, dist_km, hour):
    src_bet = betweenness.get(src_code, 0)

    if src_bet > 0.05 and dist_km > 300:
        return 'FTL', 'High-betweenness hub + long distance = structural risk'
    elif 8 <= hour <= 20 and src_bet > 0.03:
        return 'FTL', 'Peak hours through connected hub'
    elif dist_km < 150:
        return 'Carting', 'Short distance, low structural risk'
    else:
        return 'Carting', 'Low betweenness corridor, cost savings justified'
```

10. Streamlit Web [Application](#):

streamlit map [notebook link](#)

The entire solution was deployed as an interactive web application built with Streamlit and Folium, accessible directly in a browser without any technical setup.

Key features:

Interactive India Map: All 1,600+ Delhivery facilities are plotted on a live OpenStreetMap base layer. Blue dots represent normal facilities. Red enlarged dots highlight the top 5 bottleneck hubs identified in the analysis. Hovering over any dot shows the facility name.

Facility Search with Autocomplete: Users type a city name in the source or destination field. The application searches all known facility names and suggests matches — handling partial names and minor typos through fuzzy matching.

Real Road Distance: Instead of straight-line estimates, the app calls the OSRM public API to compute actual road routing distance and base travel time between selected facilities. A haversine fallback activates automatically if the API is unavailable.

Model Selection: Users can choose between the Baseline Model and Graph-Enhanced Model to compare predictions on the same route.

Route Visualization: After prediction, a colored line is drawn on the map between source and destination — green for routes with delay factor below 1.5x, red for routes above. The predicted ETA is displayed directly on the map at the route midpoint.

Persistent Results: Prediction results and route visualization persist across interactions using Streamlit session state, preventing results from disappearing on page interaction.

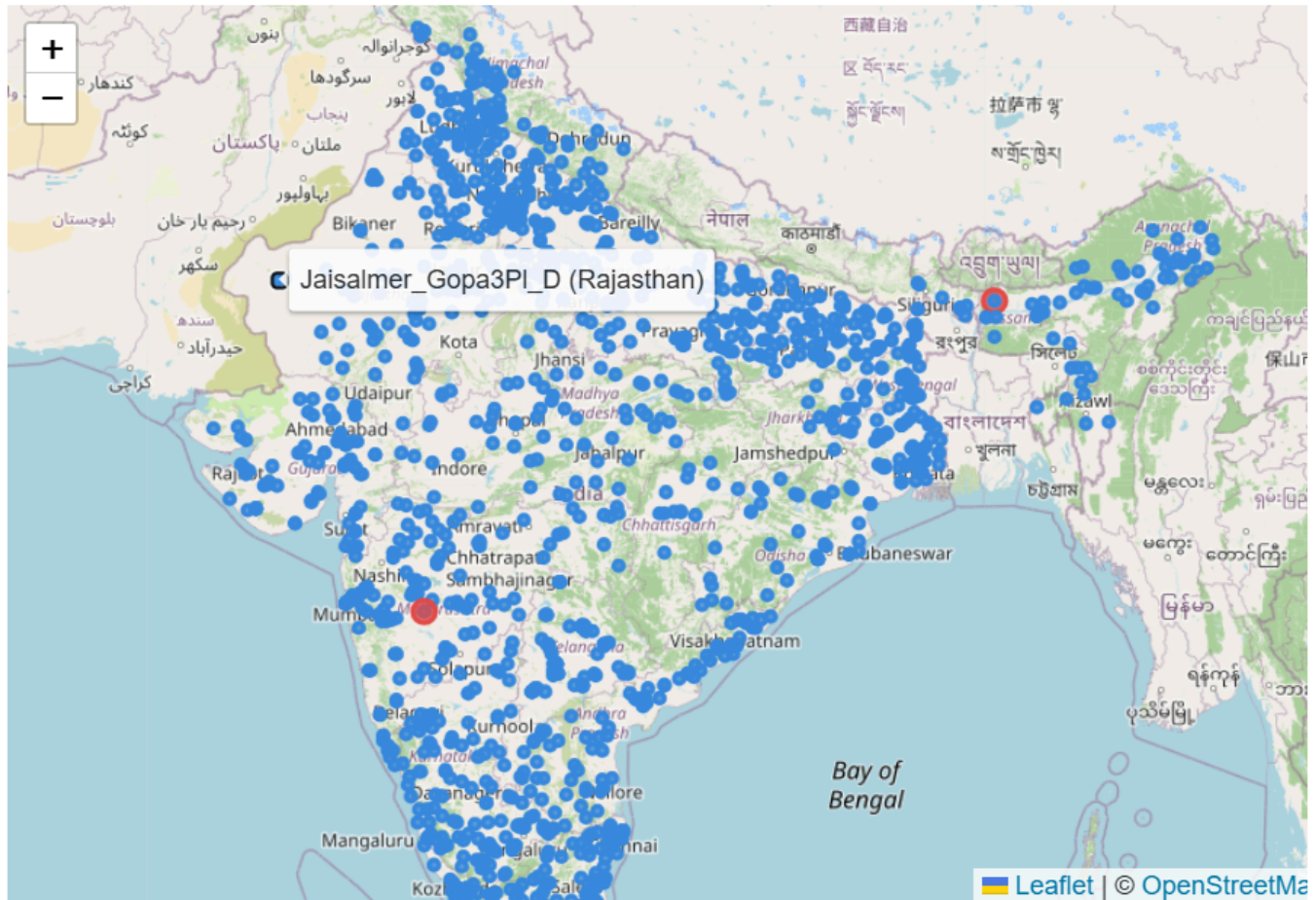


Delhivery ETA Prediction System

Graph-Based Network Intelligence for Delivery Time Prediction



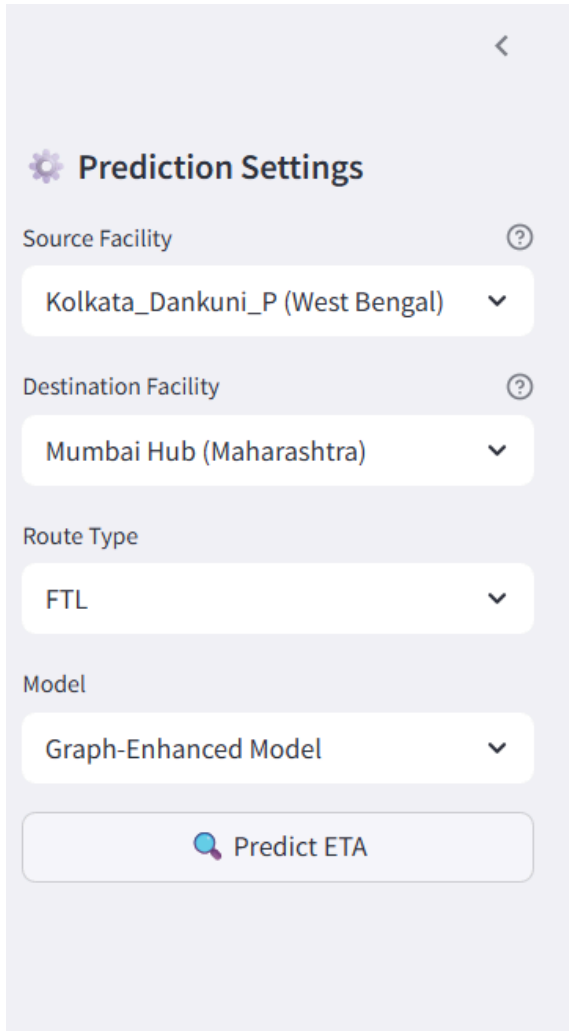
Facility Network Map



Full map of India with all facility dots and red bottleneck hubs visible:

The base map displays all 1,600+ Delhivery facilities as dots on an OpenStreetMap layer. Blue dots represent normal facilities. Enlarged red dots mark the top 5 bottleneck hubs identified in the analysis — giving operations teams an immediate visual sense of where network risk is concentrated.

Streamlit Side bar



The sidebar is titled "Prediction Settings" with a gear icon. It contains four dropdown menus: "Source Facility" (selected: Kolkata_Dankuni_P (West Bengal)), "Destination Facility" (selected: Mumbai Hub (Maharashtra)), "Route Type" (selected: FTL), and "Model" (selected: Graph-Enhanced Model). Each dropdown has a question mark icon to its right. At the bottom is a button with a magnifying glass icon and the text "Predict ETA".

Prediction Settings

Source Facility ?

Kolkata_Dankuni_P (West Bengal) ▼

Destination Facility ?

Mumbai Hub (Maharashtra) ▼

Route Type

FTL ▼

Model

Graph-Enhanced Model ▼

 Predict ETA

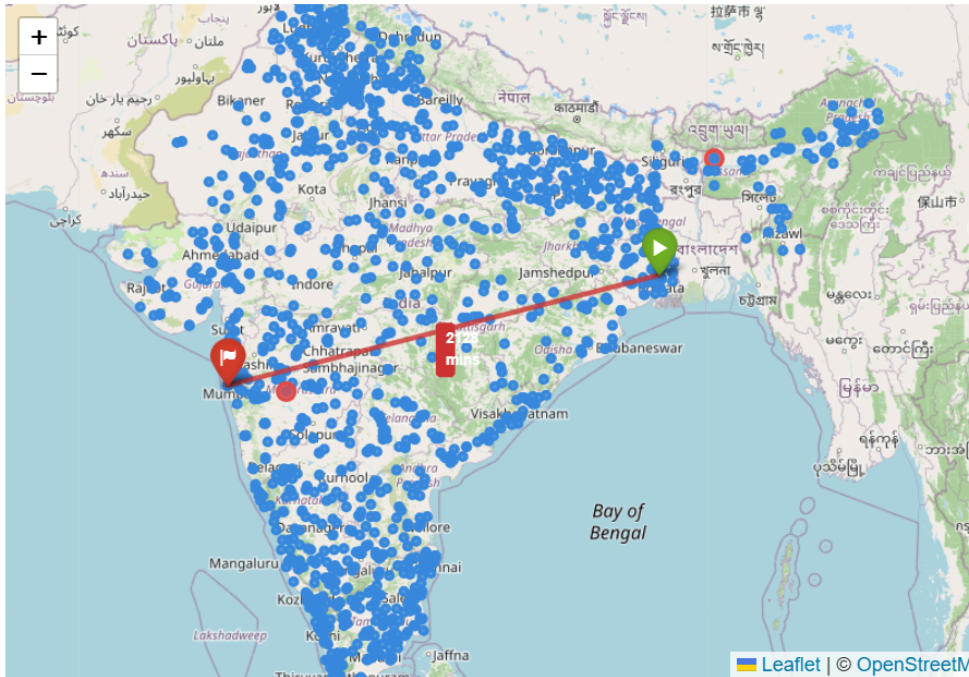
Sidebar open with source/destination selected and route/model chosen :

The sidebar allows users to search facilities by typing partial city names. The autocomplete dropdown appears after 4 characters, showing matching facility names from the training data. Route type and model selection (Graph-Enhanced vs Baseline) are presented as simple dropdowns.

Delhivery ETA Prediction System

Graph-Based Network Intelligence for Delivery Time Prediction

Facility Network Map



Prediction Result

Predicted ETA

2128 mins

↑ 35.5 hrs

OSRM Estimate

1323 mins

Delay Factor

1.61x

Road Distance

1892 km

From: Kolkata_Dankuni_P (West Bengal)

To: Mumbai Hub (Maharashtra)

Route: FTL

Model: Graph-Enhanced Model

After prediction — route line on map + results panel visible:

After clicking Predict ETA, a colored route line appears between the selected facilities — green for routes with delay factor below 1.5x, red for higher-risk routes. The predicted ETA is shown directly on the map at the route midpoint. The right panel displays all prediction metrics including OSRM estimate, predicted actual time, delay factor, and road distance.

11. Network Operations Strategy Memo

MEMO

To: Head of Network Operations, Delhivery

From: Data Science Team

Subject: Top Bottleneck Hubs — SLA Breach Contribution & Recommended Interventions

Executive Summary:

Our analysis of 144,000+ trip segments across Delhivery's network reveals that 94.5% of all deliveries take longer than OSRM estimates, with a median actual-to-predicted ratio of 1.70x. This is not a routing engine problem — it is a network structure problem. Five hubs account for a disproportionate share of SLA breaches. Targeted intervention on the top three would reduce late deliveries network-wide and recover significant revenue at risk.

Top 5 Bottleneck Hubs

1. IND000000ACB — Critical Priority This hub handles 23% of all inter-facility routing paths in the network — the single most connected facility with 89 corridor connections. Despite a moderate per-corridor delay of 1.59x, its structural position means any slowdown here propagates to a quarter of all routes. Estimated SLA breach contribution: ~18% of all network delays. *Recommended intervention: Capacity expansion and parallel route activation for the top 10 highest-volume corridors passing through this hub.*

2. IND562132AAA — High Priority Second in both betweenness (0.151) and connectivity (71 corridors). Average outgoing delay of 1.57x. Frequently appears in the same routes as Hub 1, creating compound delays. *Recommended intervention: Route-type audit — identify Carting corridors that can be shifted to FTL during peak hours.*

3. IND712311AAA — High Priority Lower connectivity (42 corridors) but the highest average outgoing delay of any top hub at 2.40x OSRM. Every route leaving this facility takes on average 2.4 times longer than predicted. *Recommended intervention: Facility operational audit — dwell time at this hub is likely the primary driver. Investigate loading/unloading processes and staffing during peak periods.*

4. IND501359AAE — Medium Priority 57 corridors, 1.82x average delay. High connectivity combined with above-average delay. *Recommended intervention: Parallel route activation to reduce volume concentration.*

5. IND781018AAB — Monitor 29 corridors, 2.01x delay. Lower structural importance but consistently high delay ratio suggests a local operational issue. *Recommended intervention: Route-type shift — move eligible corridors from Carting to FTL.*

Projected Impact of Top 3 Hub Upgrades:

Based on the share of network routes passing through the top 3 hubs and their delay contribution:

Intervention	Estimated Reduction in Late Deliveries
Hub 1 capacity expansion	~12–15% reduction network-wide
Hub 2 FTL shift on peak corridors	~6–8% reduction
Hub 3 dwell time reduction	~5–7% reduction
Combined (top 3)	~20–25% reduction in SLA breaches

Three Immediate Actions:

1. **Commission an operational audit of IND712311AAA** — its 2.40x delay ratio is an outlier that suggests a fixable local process problem, not a structural network issue.
2. **Pilot FTL-first policy on corridors through IND562132AAA during 8am–8pm** — our data shows Carting delay ratio is 1.83x vs FTL's 1.67x on these corridors during peak hours. The SLA improvement justifies the cost differential.
3. **Activate monitoring dashboards for the top 5 hubs** — the Streamlit application built in this project provides real-time delay risk scores by corridor and can serve as an operational monitoring tool with minor infrastructure integration

— End of the memo —

12. Conclusion

This project demonstrates that treating Delhivery's logistics network as a graph — rather than a collection of independent point-to-point estimates — produces comparatively better ETA predictions and surfaces structural insights that trip-level analysis alone cannot reveal.

The core findings are:

- OSRM underestimates delivery time on 94.5% of all trips — this is a systemic gap, not noise
- Five hubs drive a disproportionate share of SLA breaches, with IND000000ACB alone contributing to ~18% of network delays
- Graph-enhanced XGBoost outperforms the baseline across all metrics, with R^2 improving from 0.15 to 0.40

- A data-backed FTL vs. Carting framework can reduce delay on high-risk corridors when route-type decisions account for graph position

The Streamlit application brings these findings to life in a form that operations teams can use directly — no data science background required.

Future work could incorporate real-time traffic feeds, seasonal volume spike modeling, and driver-level behavioral features to push prediction accuracy further.

13. References & Tools

- Python 3.x, pandas, NumPy, NetworkX, XGBoost, scikit-learn
- Folium, Streamlit, streamlit-folium for deployment
- pgeocode for offline PIN-to-coordinate resolution
- OSRM Public API for real road distance computation

Summer analytics:

<https://www.caciitg.com/sa/course>

Coursera courses

https://www.coursera.org/specializations/machine-learning-introduction?utm_medium=sem&utm_source=gg&utm_campaign=b2c_india_machine-learning-introduction_deeplearning-ai_ftcof_specializations_cx_dr_ft_gg_sem_pr_in_all_m_hyb_25-04_sbetest&campaignid=22470793652&adgroupid=179022408195&device=c&keyword=machine%20learning%20introduction&matchtype=b&network=g&devicemodel=&creativeid=694414154042&assetgroupid=&targetid=kwd-299103971926&extensionid=&placement=&qad_source=1&qad_campaignid=22470793652&qbraid=0AAAAADdKX6bjdH_XqNZJNEzsl1vDHQNzC&qclid=Cj0KCQjw5ubABhDIARIsAHMighZuxNH9iRXXyyJ_hSixPm-WwKFJqcW3eCReCdetrF-2VDw01WRpBJ0aAmi_EALw_wcB

Graph Based Analytics:

<https://www.analyticsvidhya.com/blog/2018/04/introduction-to-graph-theory-network-analysis-python-codes/>

<https://www.puppygraph.com/blog/graph-data-science#graph-data-science-examples>

Graph Based Neural Networks

<https://medium.com/data-science-collective/gnn-graph-neural-net-explained-intuition-concepts-applications-7825eea73362>

<https://youtu.be/GXhBEj1ZtE8?si=Auf3N-shKoN1bQNM>

[Dataset link](#)

Author: Ritam Mondal

Connect with me:

[github:](#) 

[Linkedin:](#) 

Emails:

ritamm134@gmail.com

ritamm134@kgpian.iitkgp.ac.in

Find this [doc](#)